SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	RASHMITHA SENTHIL KUMAR	**Reg. No.:**	192525034
Date:	30/10/2026	**Duration:**	60 minutes

Code of Conduct

*I **RASHMITHA SENTHIL KUMAR (192525034)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **RASHMITHA SENTHIL KUMAR**

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture● Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes
Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

	represented.			
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. ANALYZE SYSTEM REQUIREMENTS

1.1 Introduction and problem analysis:

Municipal distribution networks combine geographically dispersed pipes, pumps, reservoirs, valves and monitoring points. A buried leak can initially manifest as a subtle change in pressure, flow balance, acoustic propagation or soil moisture. A major rupture can develop rapidly and cause water loss, road damage and service disruption. A useful software architecture must therefore ingest continuous measurements, distinguish genuine leaks from sensor faults, localize suspected damage, and support safe intervention without assuming that every anomaly justifies valve closure.

The proposed platform addresses two coupled engineering problems: high-volume, delay-tolerant telemetry processing and safety-sensitive physical control. These problems have different failure consequences. Telemetry analytics may tolerate brief replay delays, whereas incorrect actuation may interrupt critical water supply or generate damaging pressure transients. The architecture consequently separates monitoring decisions from the final, independently validated control path.

1.2 System objectives:

The principal objectives are to collect timestamped pressure, flow, acoustic, moisture and valve-state telemetry; detect and classify abnormal conditions; estimate leak positions from acoustic arrival-time differences; coordinate verified incident isolation; recommend safe pressure setpoints; and preserve traceable records for utility review. The system should continue operating in a degraded, observable manner during sensor or network outages, while refusing unsafe automated control.

1.3 Functional requirements influencing architecture:

FR-01 and FR-02 require authenticated telemetry ingestion, event identifiers, timestamps, schema validation and quality checks. FR-03 requires a multi-sensor detector that compares pressure trends, flow imbalance and supporting measurements before declaring a verified incident. FR-04 requires an acoustic localization service that knows pipe length, calibrated wave speed and synchronized arrival times. FR-05 requires a valve-isolation planner and an independent safety policy gate. FR-06 requires pressure-demand analysis and bounded setpoint recommendations. FR-07 and FR-08 require dashboards, incident notifications and operational acknowledgements. FR-09 requires sensor-health monitoring, durable edge buffering and replay. FR-10 requires an audit service recording observations, decisions, commands and feedback.

These functions suggest independent processing responsibilities rather than one large request-response application. Sensor events are frequent and asynchronous, while operator interactions and control approvals are relatively infrequent but must be authenticated and auditable.

1.4 Non-functional requirements and quality drivers:

Scalability is driven by thousands of low-power devices producing measurements at different rates. Reliability requires bounded buffering, durable messaging, idempotent consumers and explicit handling of incomplete or late events. Performance is measured by detection latency, localization latency and command acknowledgement, not merely web-page response time. Security requires device identities, encrypted transport, role-based permissions and segmentation of supervisory and control networks. Availability calls for redundant infrastructure and local safe behavior when cloud or central services are unavailable. Maintainability requires versioned schemas, independently testable modules and clear responsibility boundaries.

The scenario phrases “zero data loss” and “near-zero flooding” express ambitions, not unconditional guarantees. Finite buffers, physical failures, communication partitions and hydraulic effects mean that production acceptance criteria must specify supported outage duration, telemetry retention, acceptable loss, detection latency, false-positive rate and safe isolation response. No numerical service-level targets are asserted as achieved by the academic prototype.

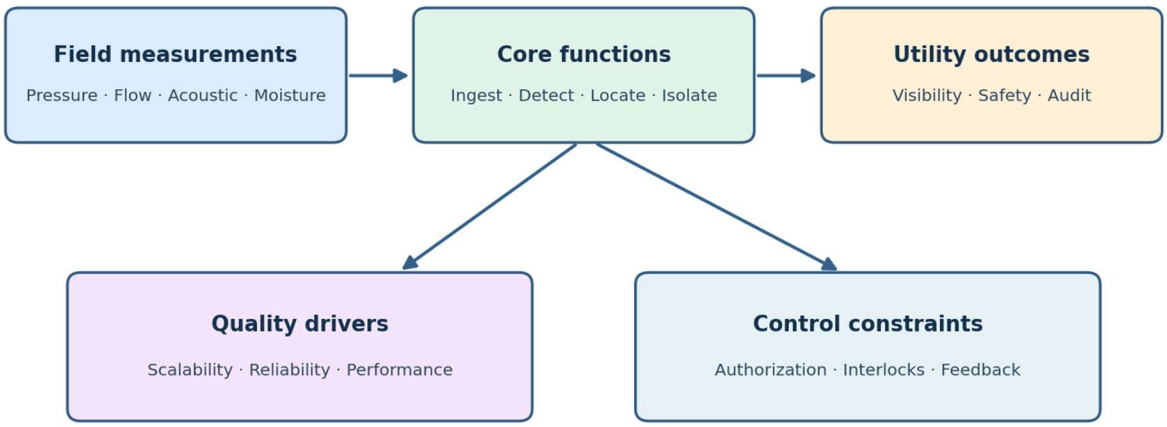


Figure 1. Requirement drivers and safety constraints for the proposed aqueduct platform.

2. SELECT A SUITABLE SOFTWARE ARCHITECTURE

2.1 Selected architecture: hybrid event-driven, layered services:

A hybrid event-driven microservices architecture with a separate safety-control boundary is selected. Field devices report to district edge gateways. Gateways validate and temporarily persist events before forwarding them to a durable event broker. Independent analytics services subscribe to the event stream to evaluate anomalies, acoustic timing, network topology and incident severity. A distinct operations layer proposes valve isolation or pressure adjustments, while a protected control gateway enforces authorization and safety rules before a programmable controller or simulator receives a command. An authenticated dashboard exposes incidents, trends and audit history.

The design is hybrid because it deliberately combines asynchronous publish-subscribe messaging for telemetry, request-response APIs for operator interactions, and deterministic local interlock logic for control. Layering provides a readable separation of field, ingestion, analytics, operations, persistence and presentation concerns. Service decomposition permits scaling the highest-volume processing paths without multiplying safety-critical command components.

2.2 Why this selection fits the scenario:

An event broker decouples bursty field ingestion from computationally expensive acoustic analysis. Partitioning by district or pipe segment allows consumer groups to process independent event streams in parallel. Persistent edge queues reduce the effect of intermittent low-power network connections. The incident service can reconstruct evidence from immutable event identifiers, and the dashboard can subscribe to derived status rather than querying every sensor directly. The isolation service remains separate from the detector so that a mistaken classification cannot directly bypass interlocks.

2.3 Alternatives and architectural trade-offs:

A conventional layered monolith would be straightforward to implement and debug but would couple high-rate ingestion, analytics and user requests in one deployment unit. Pure microservices without durable messaging would still struggle with temporary sensor disconnects and event replay. A purely serverless design could reduce infrastructure administration but introduces platform-specific latency and execution limits that complicate deterministic control paths. A pure client-server design is adequate for a dashboard but insufficient as the organizing model for asynchronous sensor networks. The chosen hybrid has greater operational complexity, requiring broker monitoring, distributed tracing, schema governance and disciplined service ownership; those costs are justified by the different scaling and safety needs of the scenario.

3. DRAW THE SOFTWARE ARCHITECTURE DIAGRAM

3.1 Overall logical architecture:

Figure 2 shows the proposed components, principal communication protocols and control boundary. It distinguishes the telemetry data plane from the safety-critical command plane. The physical valve controller is an external system, not a component assumed to be present in the current Python demonstration.

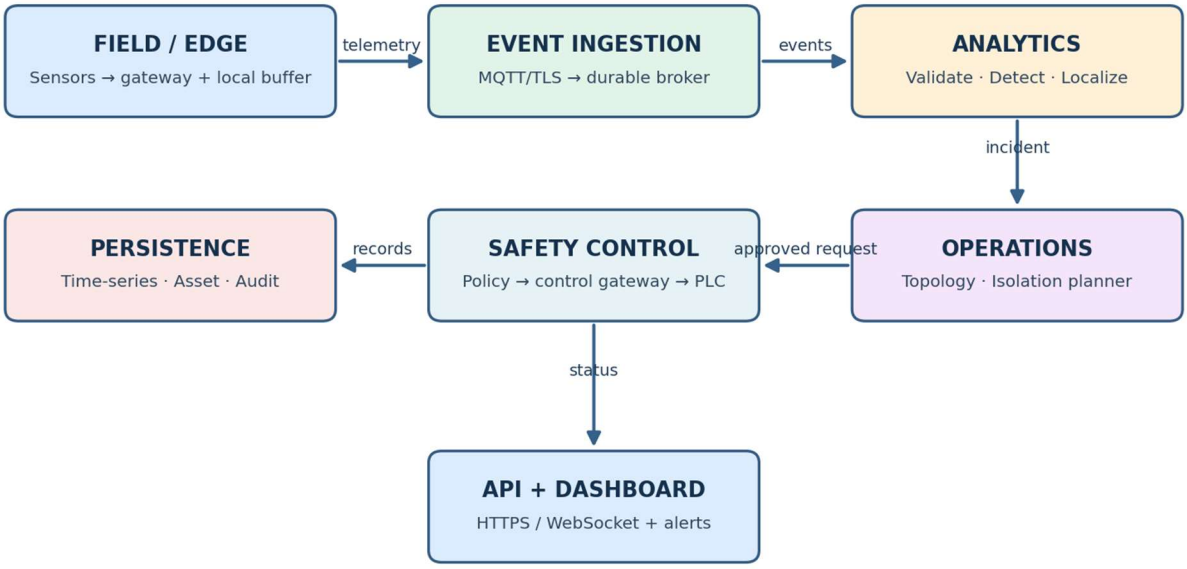


Figure 2. Proposed hybrid event-driven architecture, interfaces and independent control boundary.

3.2 Communication, storage and trust boundaries:

Gateways communicate with ingestion services using authenticated MQTT over TLS where supported, with HTTPS as a possible alternative. The broker distributes schema-versioned events to consumers; REST or gRPC may connect internal operational services where synchronous responses are necessary. Operators access dashboard APIs over HTTPS and receive incident updates through WebSocket or server-sent events. Time-series storage holds sensor history, a relational or graph-backed asset repository stores pipe topology and valve dependencies, and an incident database stores investigations and decisions. An append-only audit trail records linked event hashes; its integrity must be protected by external access controls and independent backups, since a local hash chain alone is not immutable against privileged modification.

A network trust boundary separates utility operational technology from internet-facing application services. Only a restricted command gateway can reach approved valve controllers. Telemetry events cannot themselves execute commands. Operator approvals, incident evidence, safety checks, command expiration, controller acknowledgement and measured valve feedback are independently recorded.

4. EXPLAIN COMPONENTS AND THEIR INTERACTIONS

4.1 Component responsibilities:

The sensor layer measures pressure, inlet and outlet flow, acoustic arrival times, soil moisture and valve position. The edge gateway authenticates devices, timestamps readings where possible, checks message structure, and stores bounded offline queues. The ingestion broker durably records accepted messages and delivers them to consumer groups. The validation service rejects malformed data and flags stale, duplicate or low-quality readings. The anomaly detector combines pressure deviations with flow imbalance and supporting evidence. The acoustic localizer calculates candidate leak positions from calibrated time differences, reporting uncertainty when synchronization or signal quality is insufficient.

The topology service maps sensor IDs to pipe segments, valve positions, critical customers and possible bypass routes. The isolation planner identifies candidate valve actions but does not execute them. The safety policy service checks authorization, interlocks, pressure consequences and operator overrides; the control gateway then transmits permitted commands and waits for acknowledgements. The incident service maintains state transitions such as NORMAL, SUSPECT, VERIFIED and ISOLATED. Storage and audit components support history, investigation and regulatory review, while the dashboard presents actionable information to utility operators and maintenance teams.

4.2 End-to-end monitoring and response workflow:

In normal operation, each sensor sends a timestamped measurement to its gateway. The gateway validates identity and retains a local copy until accepted downstream according to the configured delivery guarantee. The broker routes events to validation and analytics consumers. An anomaly detector evaluates pressure change, flow balance and acoustic corroboration. If evidence is incomplete, the system records a SUSPECT state and notifies the operator without isolating the pipe. If evidence meets verified-incident rules, localization and topology services determine a likely damaged segment and candidate isolation valves.

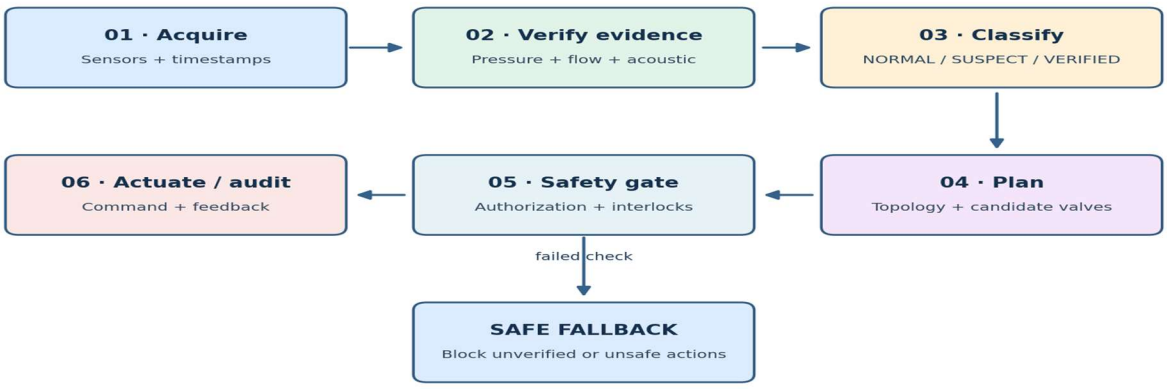


Figure 3. Proposed leak-detection, isolation and fail-safe decision workflow.

4.3 Acoustic localization and sample computation:

For two synchronized acoustic sensors separated by pipe length L, assume wave propagation speed c and define $\Delta t = tB - tA$. Under a single direct-path model, the estimated distance from sensor A is $x = (L - c\Delta t)/2$. For an illustrative 600 m segment with $c = 1000$ m/s and $\Delta t = -0.10$ s, $x = (600 + 100)/2 = 350$ m. This is a mathematical illustration, not a measured field result. In real networks, branching pipes, reflections, variable materials, uncertain wave speed and timestamp errors must be modeled or calibrated before the estimate is operationally trusted.

4.4 Fault and recovery interactions:

During a communication outage, gateways buffer events within configured storage limits and replay them after reconnection. Consumers use stable event IDs and idempotent writes to avoid duplicate incidents. If a sensor reports implausible pressure without flow or acoustic corroboration, the detector classifies the observation as unverified. If the safety interlock fails, the command is rejected and a human operator is alerted. The architecture therefore prioritizes a controlled degraded state over unsafe automatic actuation.

5. DISCUSS QUALITY ATTRIBUTES

5.1 Scalability

Partitioning telemetry by district or pipe segment allows ingestion and analytic consumers to scale horizontally. Edge filtering and event aggregation reduce unnecessary upstream traffic, while the broker isolates transient bursts from downstream service capacity. Capacity planning must include acoustic waveform sizes, event retention and peak incident traffic rather than relying only on average sensor message rates.

5.2 Security

Device certificates or equivalent strong identities, encrypted transport, least-privilege access and separated networks protect the telemetry and command paths. Sensitive commands require authenticated actors, policy checks, command identifiers, expiration and controller acknowledgement. Audit events record who authorized each action and which evidence supported it. Security testing should include spoofed devices, replayed commands, compromised gateways and unauthorized operator privileges.

5.3 Performance

The architecture reduces end-to-end latency by performing basic validation at the edge, processing event streams continuously and avoiding synchronous dashboard calls in the detection path. Performance should be measured as event-ingest lag, detection-to-alert time, acoustic localization time and verified-command acknowledgement. Thresholds should be established with utility stakeholders and tested under realistic peak loads before deployment.

5.4 Reliability

Durable broker retention, edge buffering, retry policies, dead-letter queues and idempotent event processing reduce the impact of temporary failures. Acknowledgements must be tied to the appropriate persistence boundary. The design cannot promise literal zero loss during unlimited outages or simultaneous storage failures; explicit retention and recovery limits are necessary. Fault injection should test delayed events, broker failure, duplicate telemetry and sensor clock drift.

5.5 Maintainability

Each service has a defined contract and independently testable responsibility. Versioned message schemas, automated tests, deployment pipelines, observability dashboards and rollback procedures make changes safer. Detection algorithms and safety policies should be versioned separately, with simulation and approval before production release. This reduces the risk that routine dashboard changes affect valve-control behavior.

5.6 Availability

Redundant ingestion and monitoring instances, health checks and tested recovery procedures can keep supervisory functions accessible during component failures. Local controllers must retain their certified safe behavior when central software is unreachable. Availability targets should be defined separately for monitoring, incident notification and physical control, because their recovery requirements differ.

5.7 Quality attribute verification approach

Proposed verification includes load testing with synthetic thousands-node traffic, security penetration and authorization tests, recovery testing during broker and network outages, latency measurement under burst conditions, contract tests after schema changes, and failover exercises. These are recommended acceptance activities; they are not presented as completed results of the existing Streamlit prototype.

6. JUSTIFY ARCHITECTURAL DECISIONS

6.1 Decision rationale and trade-offs

The first architectural decision is to use durable event streaming for telemetry rather than coupling sensor uploads directly to database writes and detection routines. This adds broker operational overhead but improves replayability, isolation and scale. The second decision is to separate analytic classification from valve actuation. That introduces additional control latency and complexity but prevents a single faulty sensor or algorithm from commanding physical infrastructure. The third decision is to place buffering and elementary validation at district gateways, reducing dependence on uninterrupted central connectivity while introducing a requirement for edge storage monitoring and secure device lifecycle management.

The fourth decision is to use specialized stores for different access patterns: time-series storage for measurements, topology-aware storage for network assets and a protected audit store for decisions. This increases integration work but avoids forcing all workloads into a poorly matched single database. The fifth decision is to use API-based modularity and schema versioning so that future analytical models can be introduced without rewriting field interfaces or the safety gateway.

6.2 Future enhancements and integration

Future development may integrate a calibrated hydraulic model such as EPANET, geographic information systems, supervisory control and data acquisition interfaces, district demand forecasts, predictive maintenance and digital-twin simulations. Any closed-loop pressure optimization or real-valve actuation requires hydraulic validation, operational approvals, independent safety assessment and site-specific commissioning. An open interface contract allows these systems to be connected incrementally while keeping the core incident model stable.

6.3 Prototype-to-production distinction

The academic Python/Streamlit prototype previously demonstrated synthetic major-burst alerts, a sample leak-position estimate, slow-leak and sensor-fault suspect states, simulated valve decisions and local hash-chain verification. It does not establish live MQTT ingestion, distributed broker durability, actual programmable-controller commands, validated hydraulic optimization or field reliability. Accordingly, this assignment evaluates the proposed architecture, and the existing prototype may be cited only as supplementary proof of selected simulated workflows.

6.4 Conclusion

The hybrid event-driven and layered-service architecture is technically suited to the scenario because it addresses high-rate asynchronous telemetry and safe physical intervention as separate concerns. Durable messaging and scalable analytics support wide-area monitoring, while explicit control authorization, independent interlocks and feedback checks reduce the risk of inappropriate valve isolation. The design also supports maintainability, observability

and future utility integration. Production readiness remains conditional on stakeholder-defined acceptance targets, hydraulic modeling, cybersecurity review and field testing.